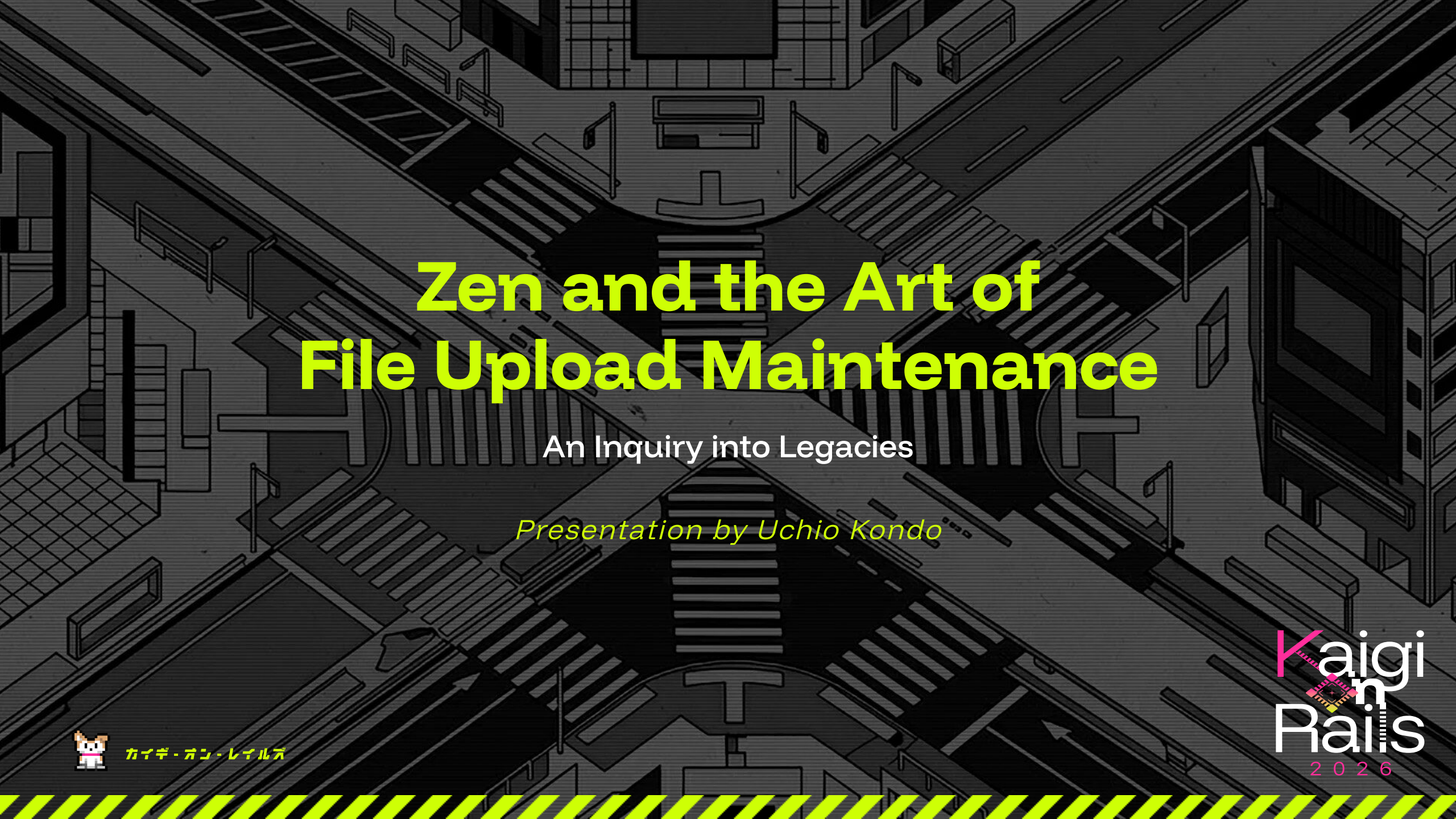

Kaigi
on
Rails
2026
カイギ・オン・レイルズ



Zen and the Art of File Upload Maintenance

An Inquiry into Legacies

Presentation by Uchio Kondo

カイギ・オン・レイルズ

Kaigi on Rails 2026

Intro

- Think back to 200X.
- You built your first blog app with Rails scaffold.
- The title and body showed up on the screen.

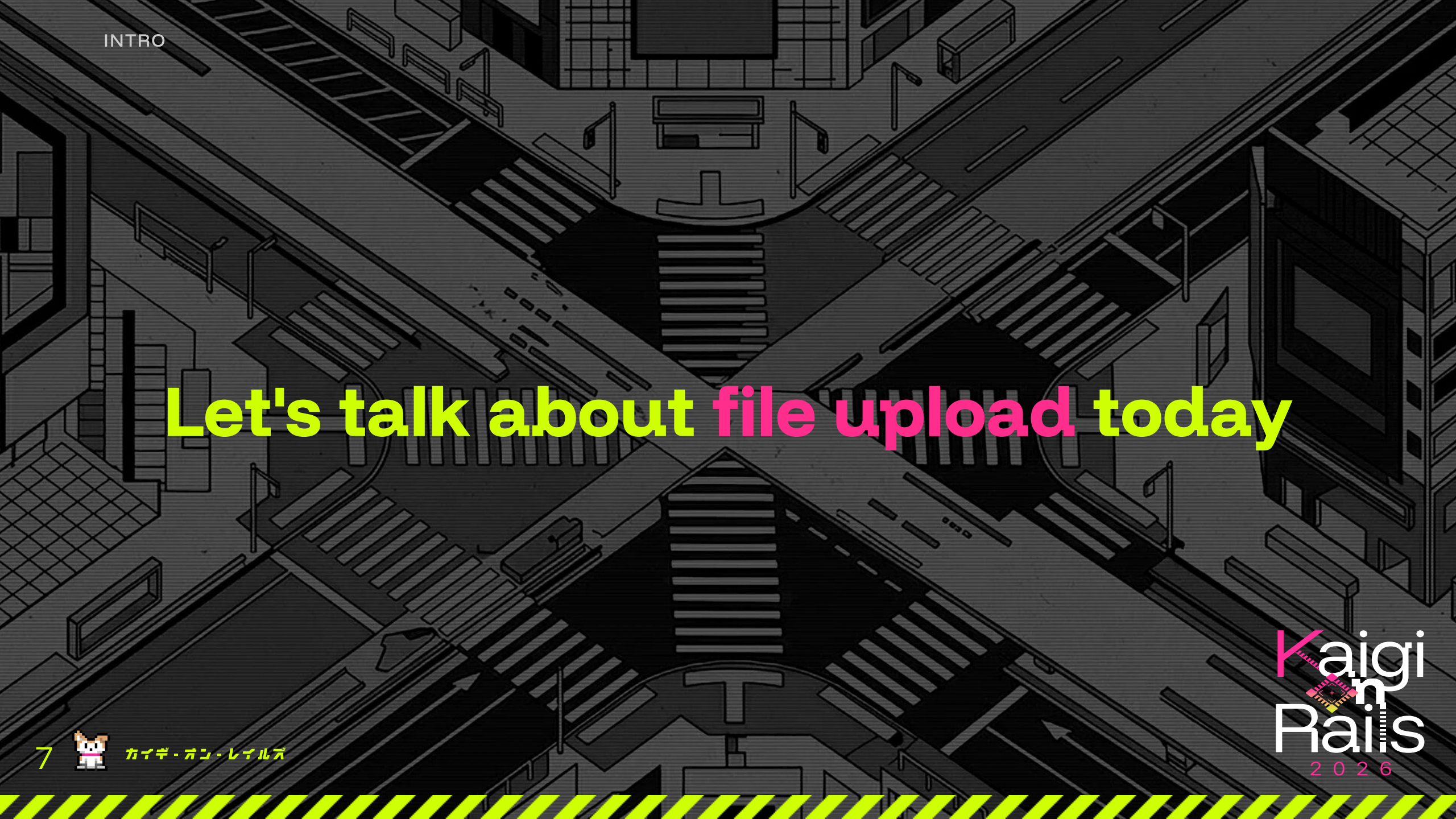
- What feature do you want to build next...?



Kaigi on Rails 2026

- Yes, **file upload**.




Let's talk about file upload today

About Me

- Uchio Kondo / @udzura
- SmartHR, Inc.
  - Technology Platform Division
  - Favorite SmartHR feature: Personnel Orders
  - Favorite Rust type: `Cell<T>`
- Fukuoka.rb / Fukuoka.wasm



Today's Route

1 Situation ▶ 2 Split the Problem ▶ 3 File Paths ▶ 4 History

▼

5 Upgrade Blockers ▶ 6 Upgrade or Replace? ▶ 7 Takeaways

Main focus: **File Paths** and **History**

Kaigi on Rails 2026

Before We Begin

- This talk is more a collection of **questions** than a single conclusion.
- Some details have been adapted; the **technical substance** is unchanged.
- Reasonable engineers may make different calls. Let's discuss them in the hallway or at the social event.

ぜひ、廊下や懇親会でお話ししましょう！

1. Situation

Rails and ActiveStorage

- One of the most basic requirements of a web service
- Today, Rails has **ActiveStorage**
  - Introduced in Rails 5.2 (2018)

B.A. (Before ActiveStorage)

- Before that, no "Rail" was laid for files
- Many gems competed
  - attachment_fu
  - Paperclip (deprecated in 2018)
  - **CarrierWave** / Dragonfly / Refile
  - Shrine

When It Was Built

- SmartHR's application got its first commit in February 2015
- A world before ActiveStorage

9 Years Later

- Migrated once from Paperclip, then about 9 years on CarrierWave
- **32** Uploader classes · **87** `mount_uploader` calls
- Up to **8** images per model
- CarrierWave is on an older version; upgrades require compatibility work
- Other dependencies, especially **Ruby and Rails**, are kept near the latest versions

2. Split the Problem

Symptoms

Fragile Path Logic

EXIF Processing

Difficult Upgrades

Synchronous Image Resizing

Implicit Uploads

Tangled Dependencies

Kaigi on Rails 2026


The problems are tangled.
Let's break them down.

Start with Smaller Problems

- “Replace CarrierWave” is too large to tackle at once
- Separate **file access**, **history behavior**, and **other dependencies**
- Choose a change small enough to **implement and verify**

Split the Problem into Three Qualities

Reliability

Unstable file paths

Recurring incidents

Keep files reachable

Usability

Slow uploads

Developer productivity

Reduce waiting and friction

Maintainability

Tangled dependencies

Difficult upgrades

Make changes easier

First, classify the problems.

Then, decide what to tackle first in our situation.

Set Priorities Across Three Qualities

1 · PRIMARY	2 · SECONDARY	3 · INITIALLY
Reliability	**Usability**	**Maintainability**
Prevent incidents Keep files reachable	Customer-reported slow uploads	Library upgrades Priority rising soon
Our main focus	**Alongside reliability work**	**Based on security needs**

This was our starting order, not a permanent ranking.

We plan to raise upgrade priority soon based on the security situation.

Kaigi on Rails 2026

Two Main Problems Today

Given our current situation, we focus on:

1. **File paths:** keep saved files **reachable**
    - Fix **fragile file paths** and make them **stable**
2. **History:** avoid needless uploads when copying records
    - Tangled history logic **slows down uploads**

Third, briefly: **maintainability** — simplify dependencies.

Kaigi on Rails 2026

3. Focus on File Paths

Why Do File Paths Matter?

- Past incidents involved **incorrect file references**
- **No files were lost** in these incidents
- All these incidents were mitigated and resolved; affected customers were informed
- **Reliability comes first** for HR documents

過去の障害はすべて止血・復旧・顧客説明済みです。

Kaigi on Rails 2026

Prerequisite: CarrierWave Lifecycle

Assign → cache!

Temporary file

→

store!

Persistent file

→

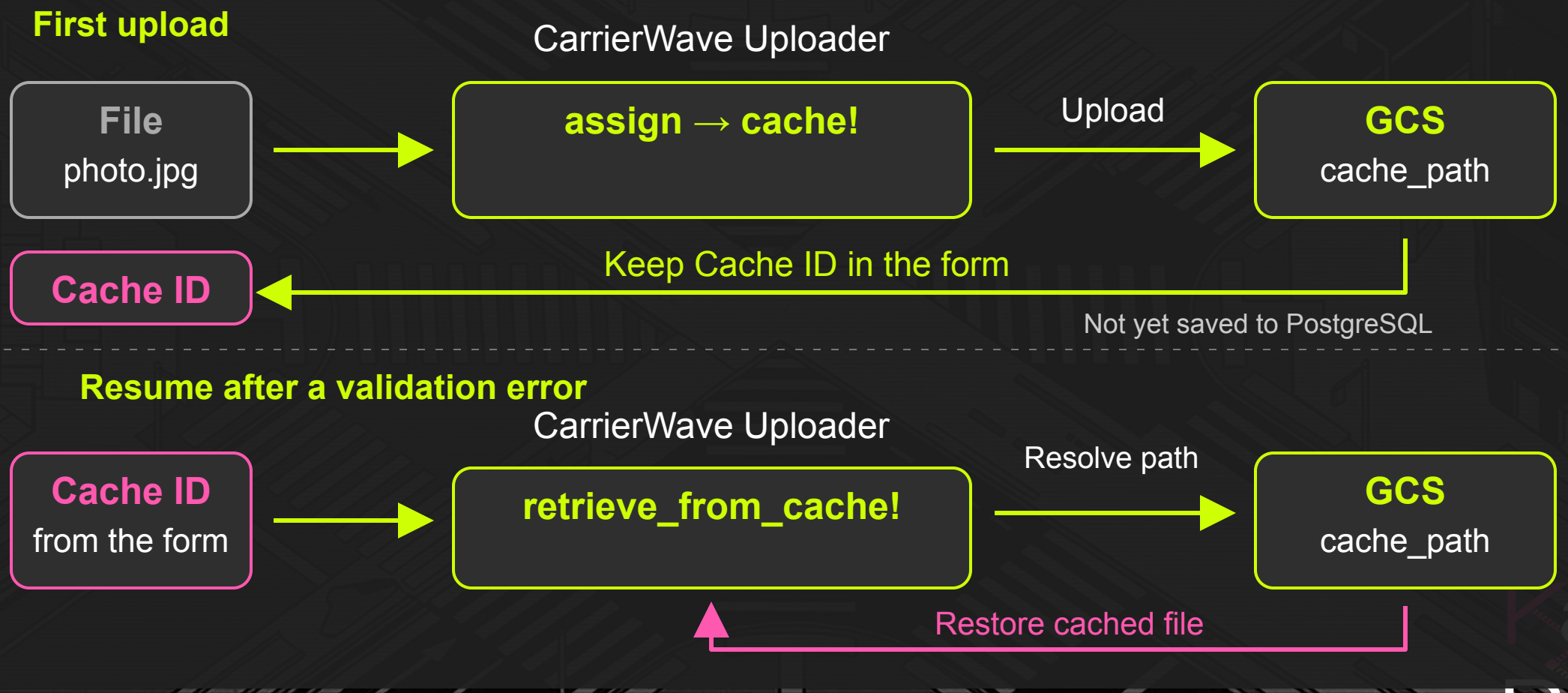
Retrieve

Access it later

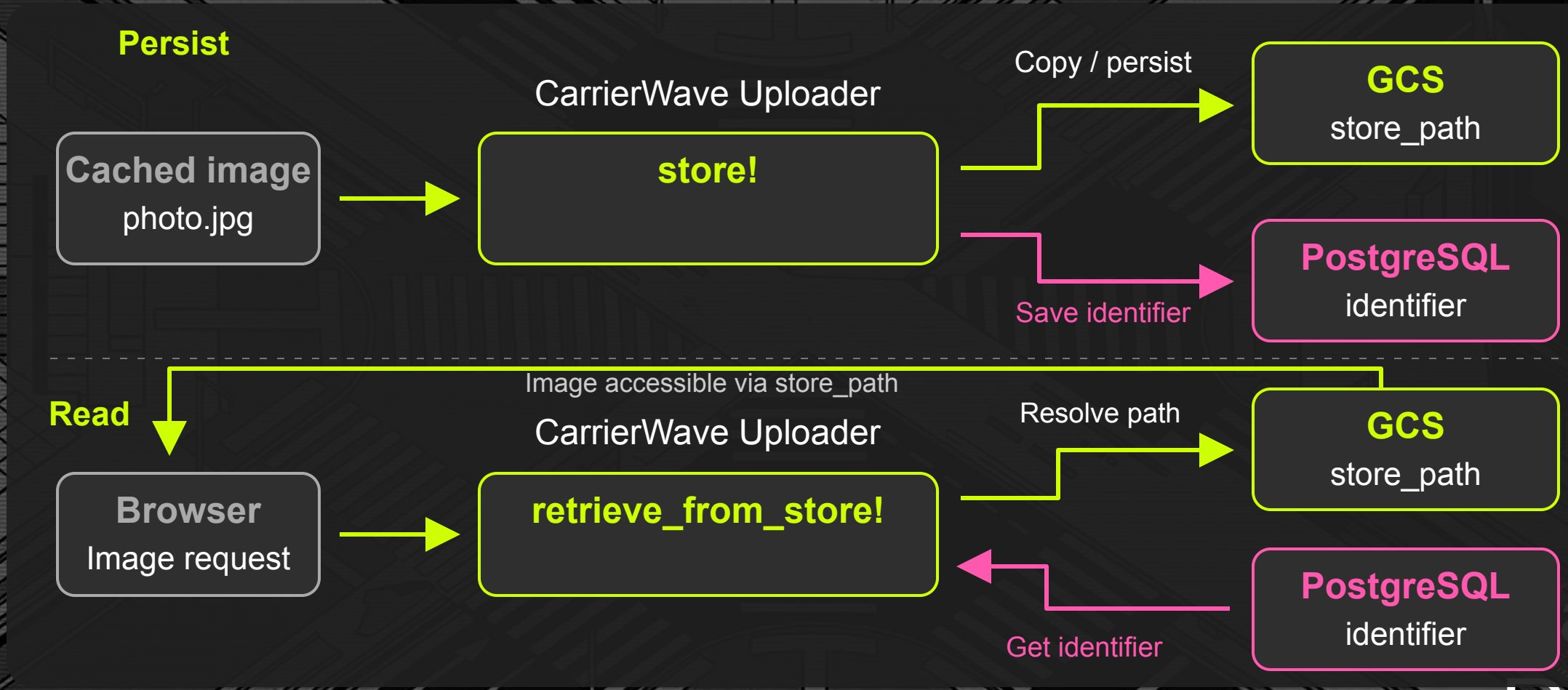
On a validation retry, a **Cache ID** can restore the temporary file.

Kaigi on Rails 2026

CarrierWave: Cache



CarrierWave: Store



igi n Rails 2026

File Paths Can Be Overridden Easily

- Convenient customization through `store_dir`
- Include model data such as an organization key or profile key
- Changes to that data or the method can make file paths **fragile**

```ruby
# Profile image example; attribute names are illustrative
def store_dir
  "profile_images/#{model.organization_key}/#{model.profile_key}"
end
```


Kaigi on Rails 2026

Complexity Leads to a Time-Based `if`

```ruby
def store_dir
  if after_hotfix?   # updated after a certain date?
    new_store_dir
  else
    legacy_store_dir # kept for older files
  end
end
```

- Paths are not kept as snapshots, so **we cannot change the logic**

Kaigi on Rails 2026

The Problem and the Two Fixes

- Changing model state or path rules can produce **incorrect file references**
- In these incidents, the files remained intact; the reference data was incorrect

Fix	Record the path when…	Keep it in…
#1 cache_path	the cache is created	Redis
#2 store_path	the file is stored	a dedicated DB column

ここまでのまとめ: 1. cacheとstoreがある 2. パスは開発者が壊しやすい

Kaigi on Rails 2026

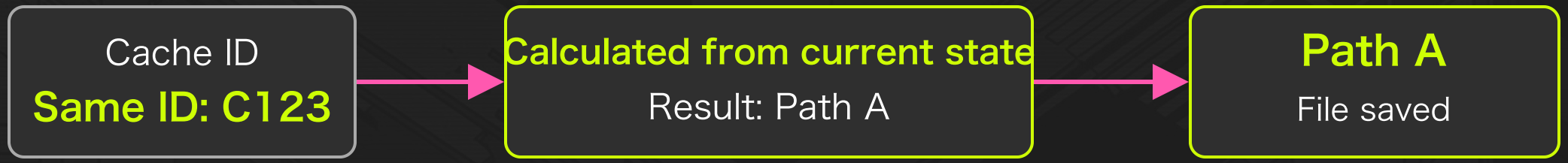
Fix #1: Record cache_path

- Between cache and store, the calculated cache path could change
  - e.g., code where the path changes depending on whether an ID exists
- This caused multiple incidents in the past
- Fix: save the path at cache time, and never recalculate it

Kaigi on Rails 2026

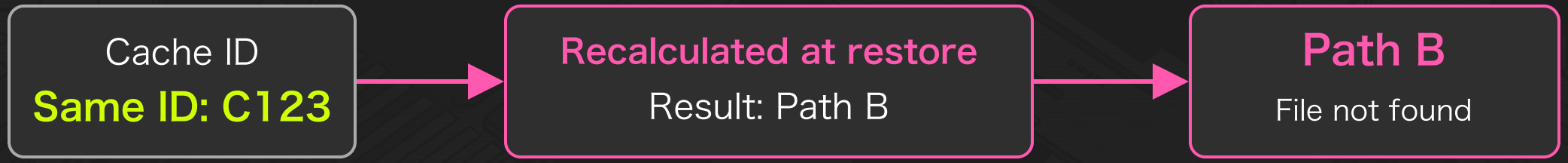
Cache: Before

① On cache creation



If the logic or inputs change in between…

② On restore



Same ID, wrong location. The file is still at Path A

Conceptual diagram: IDs and paths are examples. The ID identifies the cache, not the model.

Kaigi on Rails 2026

Cache: After

① On cache creation

Cache ID
Same ID: C123

→ **Record mapping in Redis**
C123 → Path A

→ **Path A**
File saved

Reuse the mapping

② On restore

Cache ID
Same ID: C123

→ **Read from Redis**
C123 → Path A

→ **Path A**
Same file restored

Calculate only at creation. Use the recorded path on restore

Conceptual diagram: IDs and paths are examples. The ID identifies the cache, not the model.

Kaigi on Rails 2026

Cache: Result

- Completed at the end of 2025, as our first improvement
- Incidents caused by cache_path dropped to **zero** 🎉
- Next: prevent store_path incidents from recurring

Kaigi on Rails 2026

Fix #2: Record store_path

- We could change the identifier format, but chose **a dedicated column**
  - On read, prefer that column; otherwise, calculate as before
  - Backfill existing data by recalculation
- Start with the most problematic Uploader, then expand

Store: Record the Saved Path

```ruby
class ApplicationUploader < CarrierWave::Uploader::Base
  after :store, :persist_object_store_path

  def persist_object_store_path(*)
    return if should_not_store_full_path?

    persisted_store_path = object_store_path_to_persist
    save_object_store_path(persisted_store_path)
    log_if_mismatch
  end
end
```

- Right after `store`, record the path in the dedicated column

NOTE: Simplified from the actual code.

Store: Read the Recorded Path

```ruby
class ApplicationUploader < CarrierWave::Uploader::Base
  def store_path(*args)
    calculated_store_path = super(*args)
    persisted_store_path = resolve_persisted_store_path
    log_if_mismatch
    return calculated_store_path if persisted_store_path.nil?
    use_persisted_store_path ? persisted_store_path : calculated_store_path
  end
end
```

- No record? Use the calculated path, as before
- Log any mismatch between the calculated and recorded paths

NOTE: Simplified from the actual code.

Kaigi on Rails 2026

Permanent Data Needs a Safe Rollout

- A wrong stored path can outlive the rollout
- Separate **recording paths** from **using recorded paths**
- Verify rollback as well as the forward migration

Store: Switch Writes, Then Reads

Step	write	read	Notes
1	Partly ON	OFF	Double write for some tenants. Reads as before
2	All ON	OFF	Double write for all tenants. Reads as before
3	All ON	Partly ON	Some tenants read from the fixed paths
4	All ON	All ON	All tenants read from the fixed paths

Kaigi on Rails 2026

Flipper: Roll Out and Roll Back

- Use **Flipper** to enable write/read flags for selected tenants
- Add a dedicated column alongside the existing identifier
- Include **rollback checks** in QA

Monitor the Rollout with BigQuery

- Application data was **periodically synced** to BigQuery
- We queried that copy to **monitor the file path rollout**

```sql
-- Daily check on the BigQuery copy (simplified)
SELECT id, tenant_id, avatar, avatar_object_path
FROM app_mirror.employee_profiles
WHERE
  -- a file without a path, or a path without a file
  (COALESCE(avatar, '') = '') != (avatar_object_path IS NULL)
  -- or the path does not end with the file name
  OR NOT ENDS_WITH(avatar_object_path, CONCAT('/', avatar))
```

NOTE: Simplified from the actual code.

Kaigi on Rails 2026

QA: Users Should See No Change

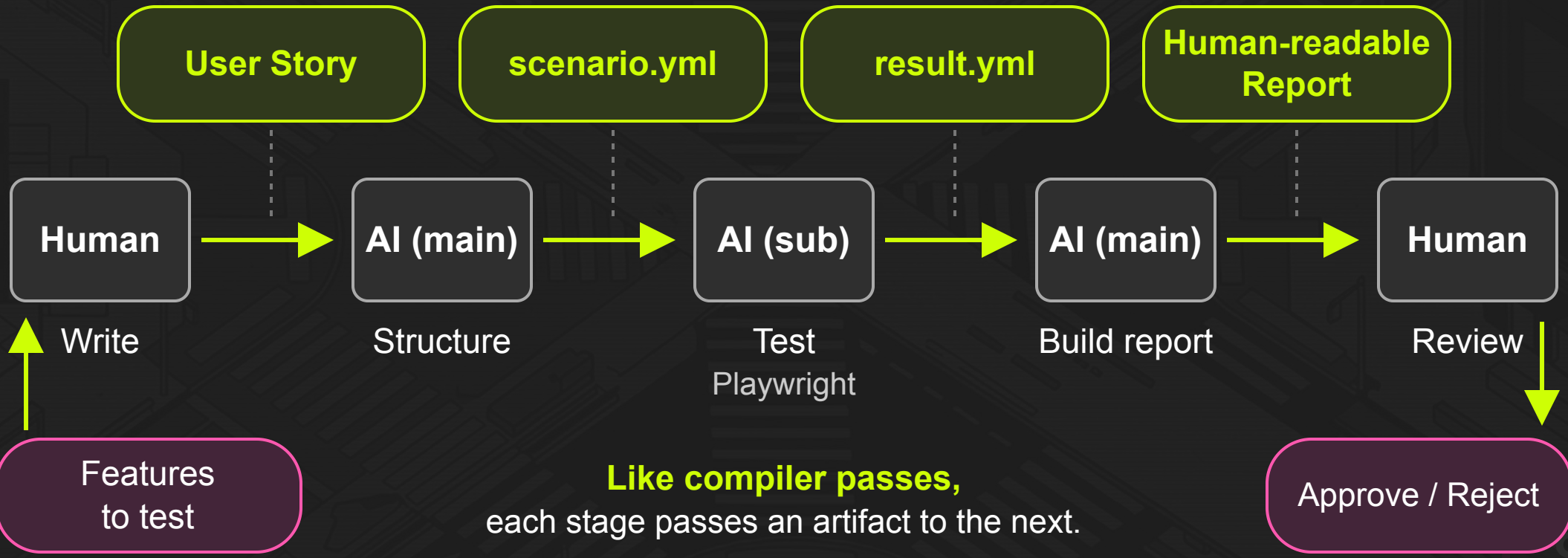
- All AI-assisted QA ran in **staging with no customer data**
- Features × preconditions create many test cases
- Define user stories and expected results with AI
- Keep automated tests and check **human-visible evidence**

AIを用いたQAはすべて、顧客情報を含まないstaging環境で実施。

Kaigi on Rails 2026

QA: Stories Become Evidence

All AI-assisted QA: staging with no customer data



kaigi on Rails 2026

QA: Example Report

- One run = one report
- Verdict, reason, and evidence for each step
  - Evidence: screenshots and logs
- Other findings go to "Warnings"



QA実行レポート

object store path の書き込みと読み込みを有効にした状態で、従業員詳細にプロフィール画像が表示される

シナリオ結果：PASS

実行概要

feature	avatar-object-path
シナリオID	AV-D-01-write-on-read-on
シナリオ名	object store path の書き込みと読み込みを有効にした状態で、従業員詳細にプロフィール画像が表示される
実行ID	20261005124743
対象リポジトリ	[redacted]
commit SHA	[redacted]
実行環境	local
開始日時	2026-10-05 12:48:06 JST
終了日時	2026-10-05 12:50:34 JST

集計

STEP 数	PASS	FAIL	ERROR	NEEDS_REVIEW
2	2	0	0	0

実行結果サマリー

STEP	操作内容	ステータス	所要時間
step-01	対象従業員の従業員編集画面（crew_edit_url）を開き、プロフィール画像に profile_image_path の画像を指定して登録する	PASS	39秒
step-02	対象従業員の従業員詳細画面（crew_detail_url）を開く	PASS	10秒

QA: Results and Human Review

- About **30 large stories**, checked in **2 days by about 3 people**
- Screenshots, logs, verdicts, and reasons are reviewed in PRs
- Existing tests are kept; automated coverage is strengthened
- The value: **more cases checked**, with a human final review

Store: Current Status

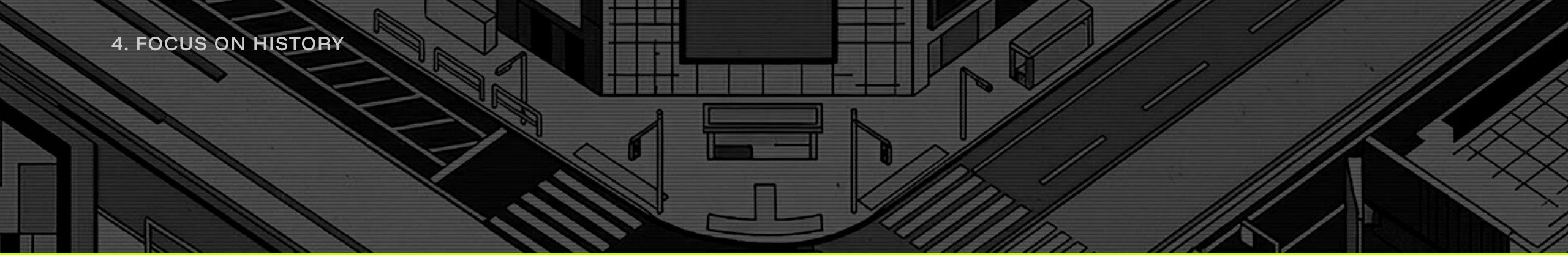
- Some images have **fully switched** to fixed paths
- We will expand to all images, step by step, the same way

File Paths: What Matters

- **Record a stable reference** to each saved file
- Separate writes and reads to make the rollout controllable
- **Test the way back**, not only the way forward

まとめ: 壊れやすいパスを1:1で引けるよう固定した。
大きな変更は必ず戻せるように、AI駆動QAで品質を担保。

Kaigi on Rails 2026



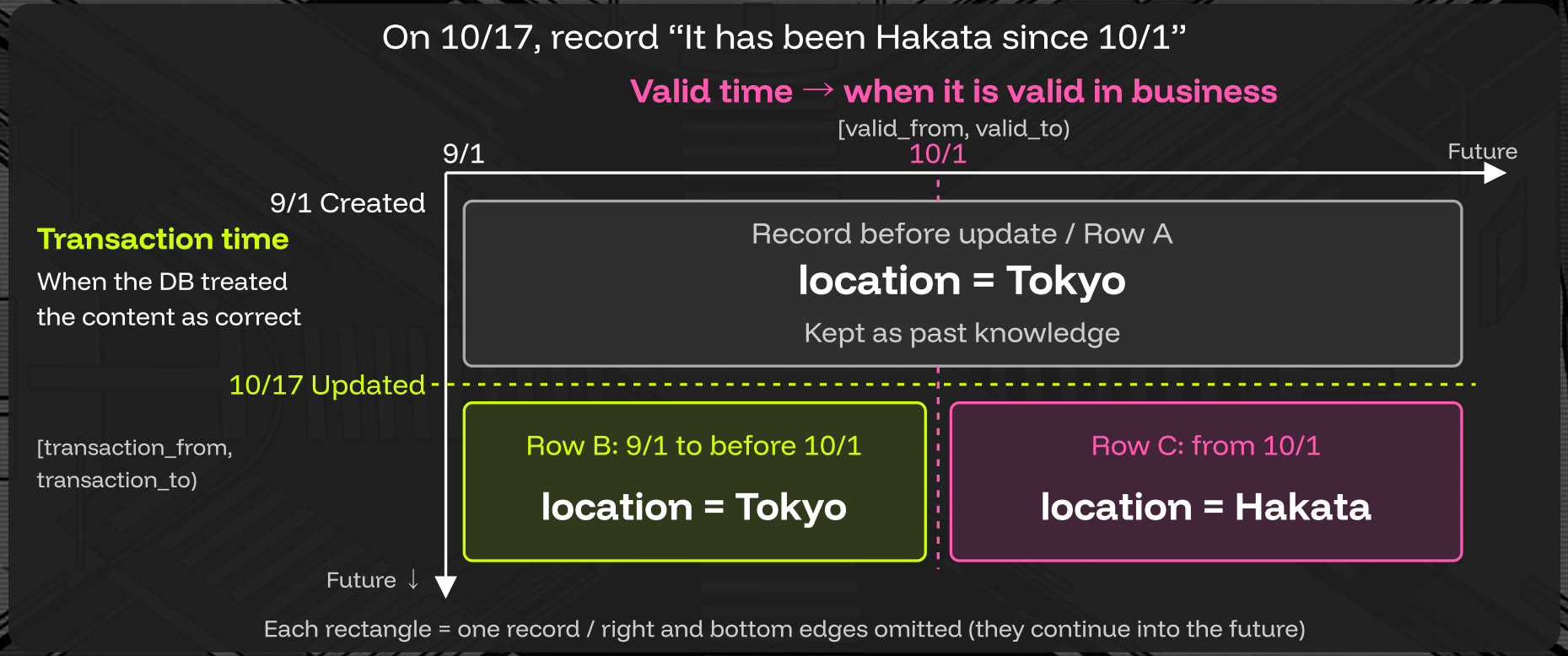
4. Focus on History



Making Images and History Work

- Employee history is essential to our service
- Handling images is essential too
- Combining them requires **extra care**

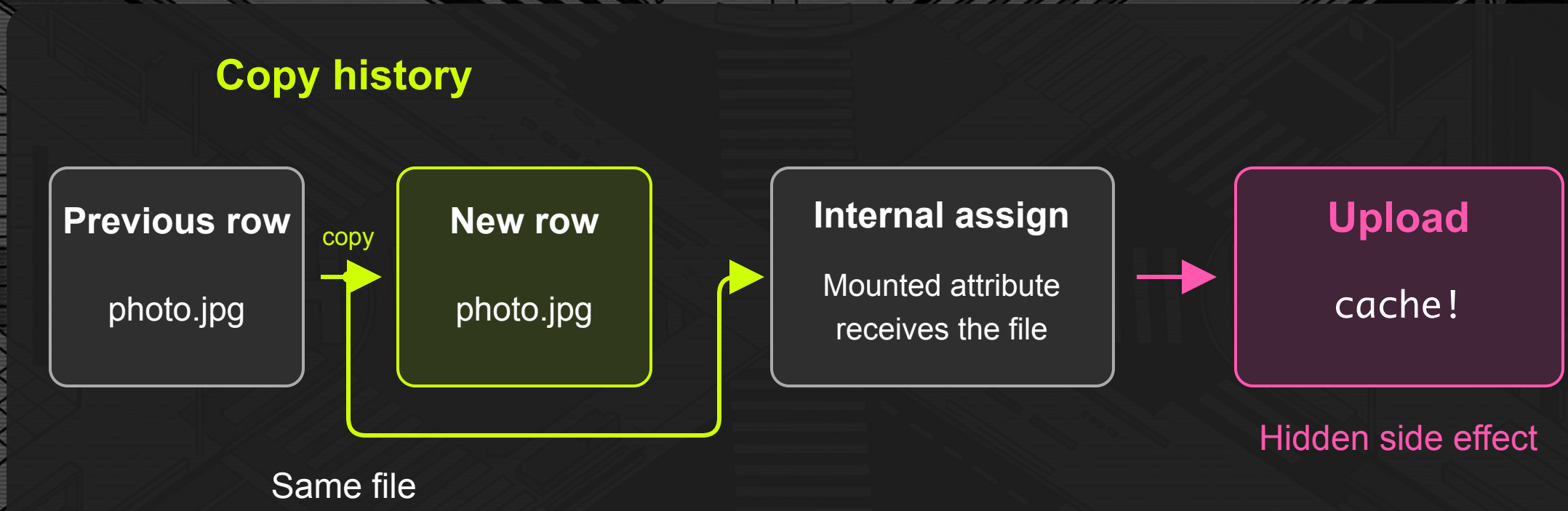
Bitemporal Data Model and Us



See also "履歴 on Rails" (KoR 2025)

Kaigi on Rails 2026

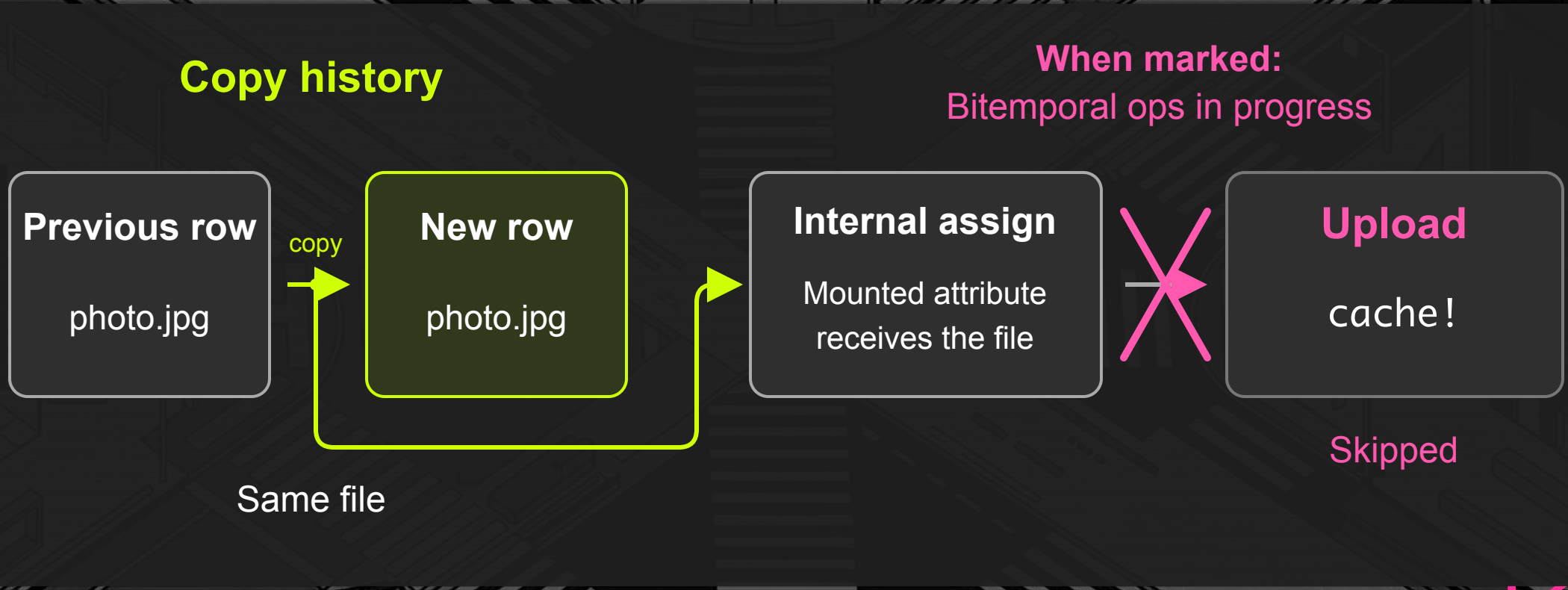
History Copying Has Side Effects



Our history-copy workflow + CarrierWave's assignment behavior can **upload the same file again.**

Kaigi on Rails 2026

The Fix: Mark History Copies



In the PR: `CurrentAttributes` carries the "copy in progress" state into hooks.

CurrentAttributes: Into the Hook

1. Mark the scope

```ruby
# Per-request state, reset by Rails
class HistoryCopy < AS::CurrentAttributes
  attribute :in_progress, default: false
end

# In ApplicationUploader
def self.copying_history
  HistoryCopy.in_progress = true
  yield
ensure # reset, even on errors
  HistoryCopy.in_progress = false
end
```

2. Skip the upload in the hook

```ruby
# In ApplicationUploader
# Assignment runs cache!; skip it during copy
def cache!(*)
  return if HistoryCopy.in_progress

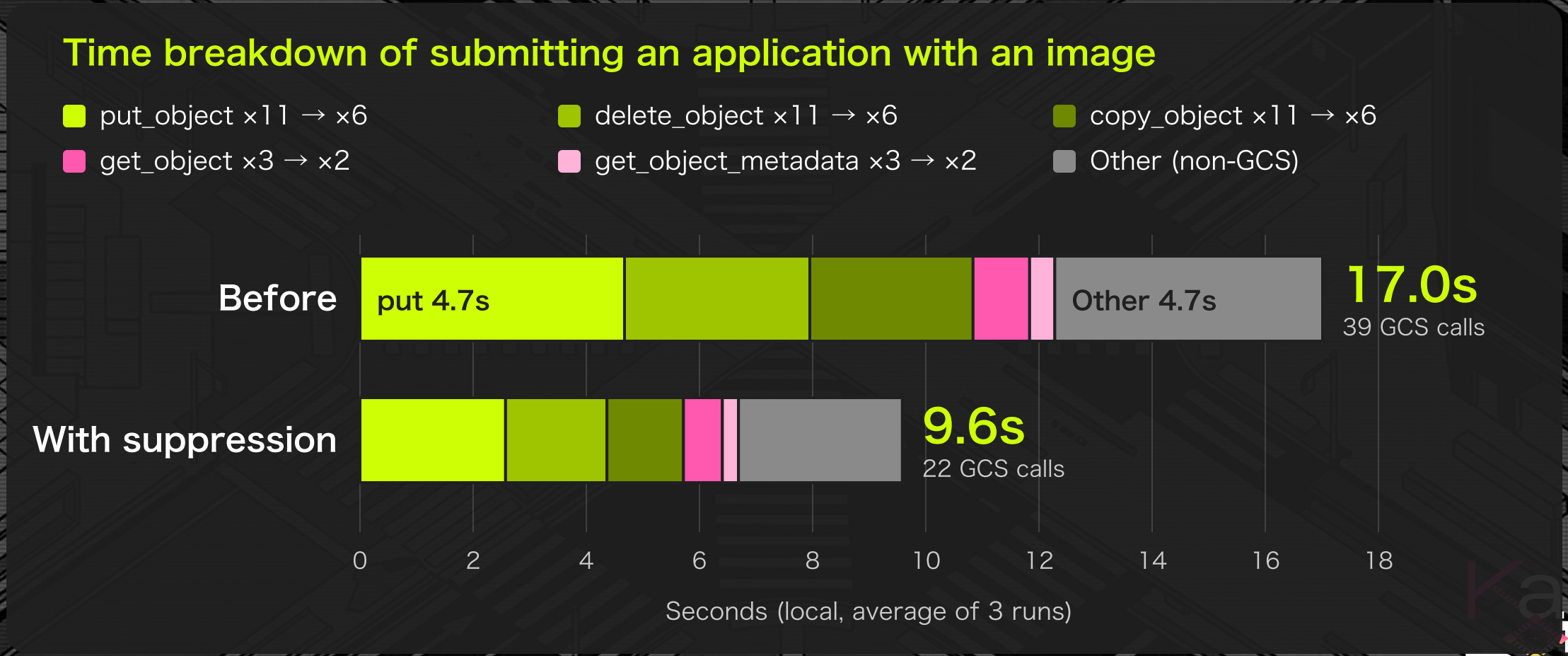
  super
end

# Usage
ApplicationUploader.copying_history do
  record.copy_history!
end
```

NOTE: Simplified from the actual code.

Kaigi on Rails 2026

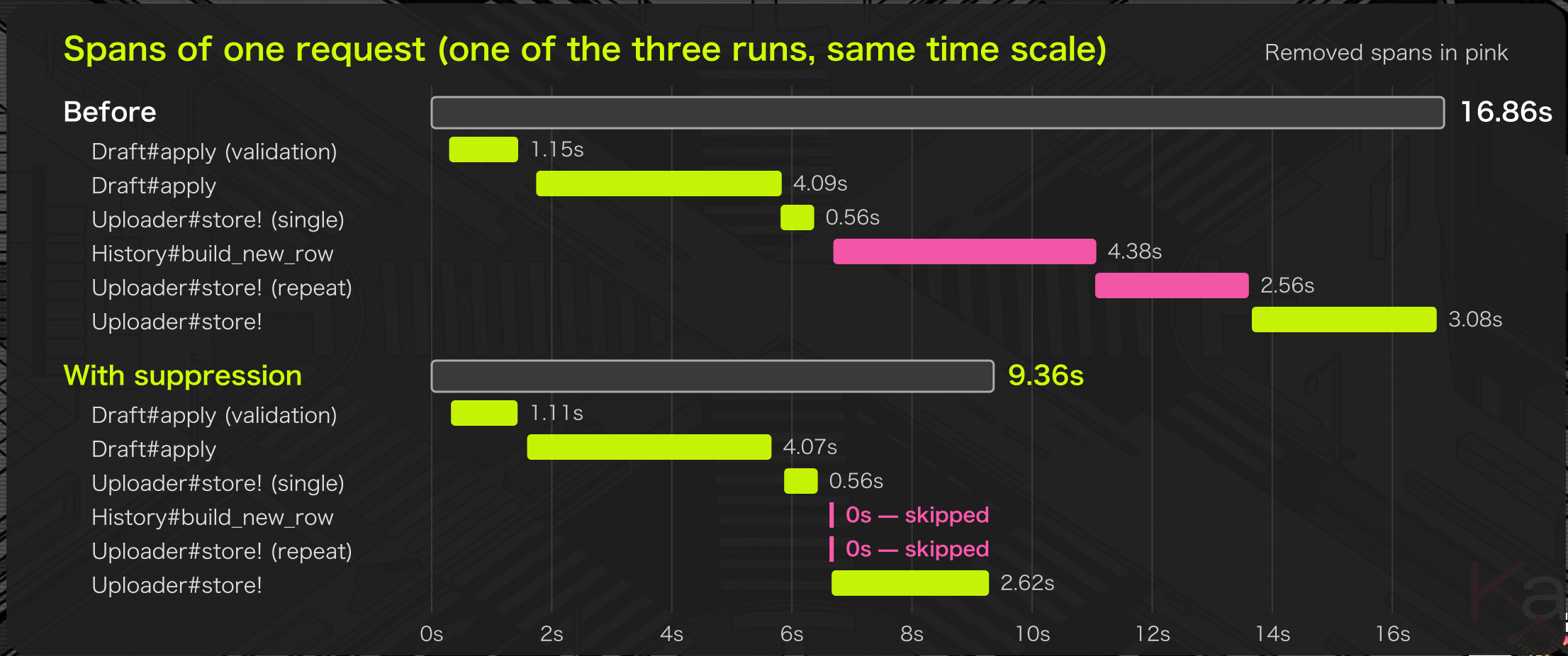
History: Measuring the Upcoming PR



Local run of a **near-release PR**: **17.0s → 9.6s (−44%)** · GCS calls **39 → 22**

Kaigi on Rails 2026

History: Where the Time Went



Two spans disappear: **rebuilding the history row** and the **repeated store**

Kaigi on Rails 2026

History: What Matters

- Make the difference between **copying a reference** and **uploading a file** explicit
- Keep lifecycle side effects under application control
- Validate the history behavior before rolling out
- The PR is nearly ready; **the production rollout is next**

まとめ: 履歴実装＋画像で不要なアップロードが起こった。
CurrentAttributesで内部フックを制御。PRは本番投入間近。

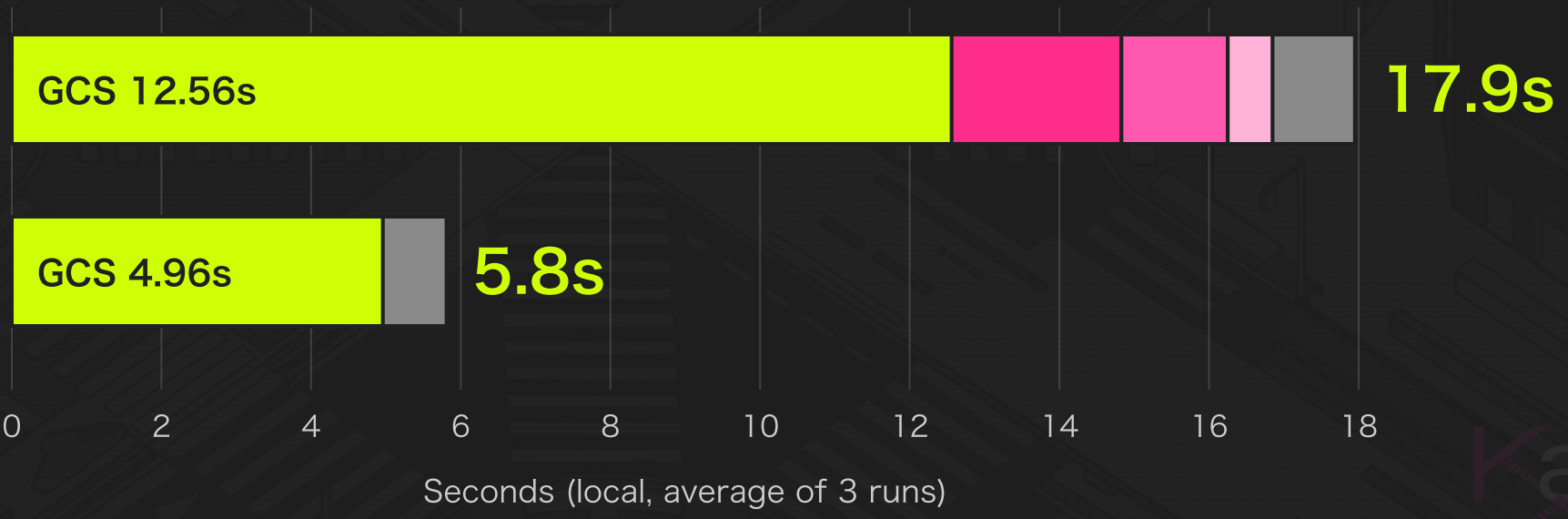
Another Performance Opportunity: Preprocessing

寄り道: 前処理にも性能改善の余地あり

Preprocessing: More Room to Improve

Time breakdown of creating an application with images

- GCS operations 12.56s → 4.96s
- Thumbnail resizing 2.27s → 0.00s
- Getting image size 1.42s → 0.00s
- EXIF operations 0.60s → 0.00s
- Other 1.10s → 0.85s



Separate local PoC: **17.9s → 5.8s** · fewer resized images, fewer GCS uploads

Kaigi on Rails 2026

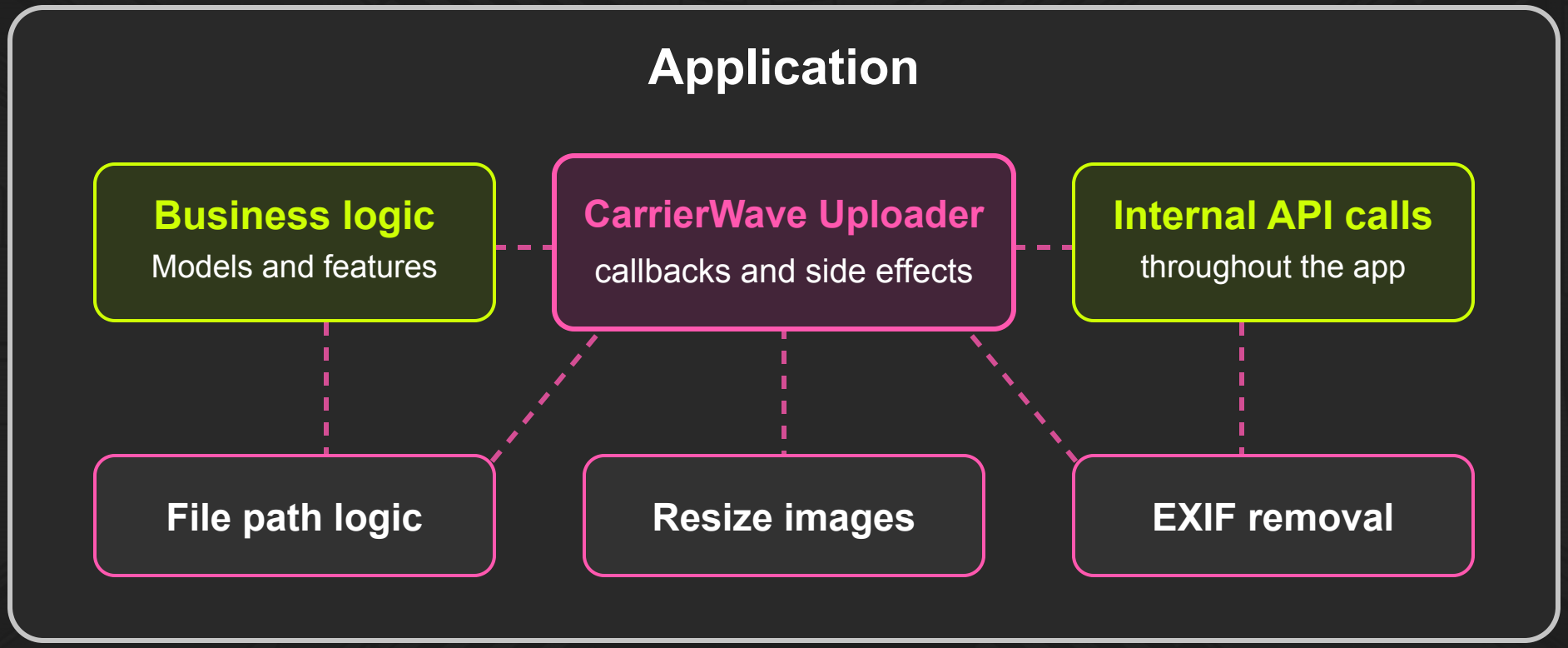
5. Upgrade Blockers

Two Blockers to Upgrading

1. **Tight coupling**
   - Application code depends on CarrierWave internals
2. **Incompatibilities**
   - A version upgrade changes behavior we rely on

更新のブロッカーは、密結合と非互換性の二つ。

Current: Coupled Image Responsibilities



Responsibilities and side effects are intertwined

Image Operations: Find the Boundary

Nearly **300 call sites** depend on CarrierWave.

Before: Uploader behavior

```
user.avatar.present?
```

Depends on the Uploader's `blank?`

After: Application intent

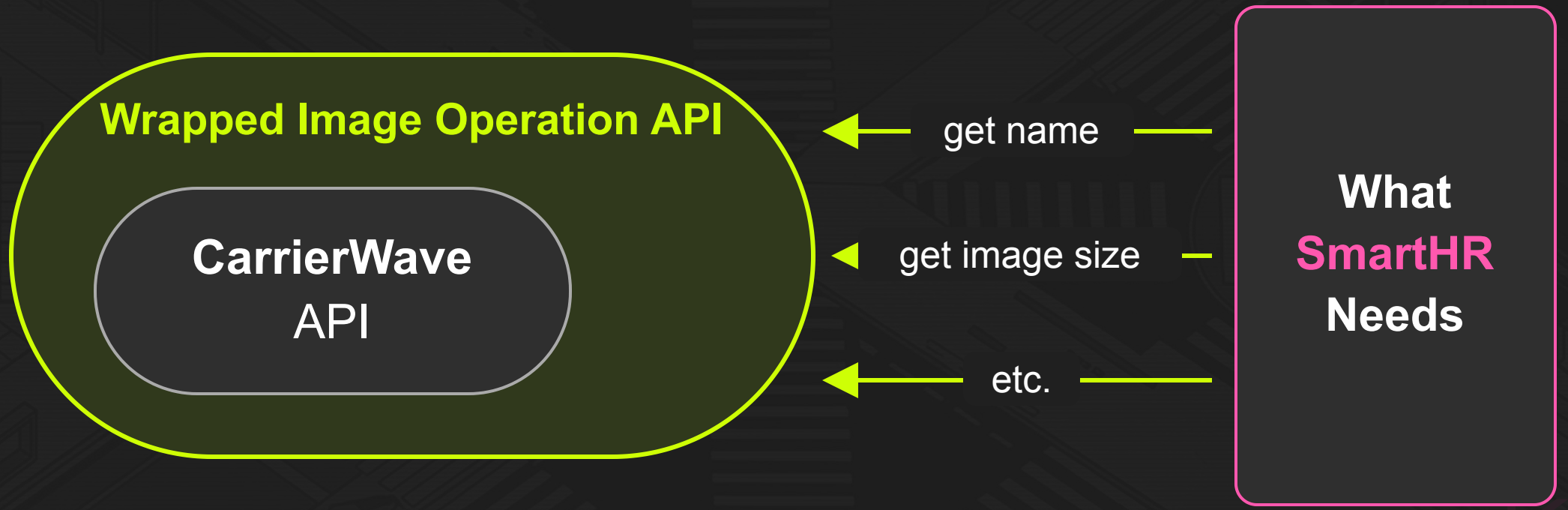
```
CarrierWaveCompatLayer.attached?(
  user, :avatar
)
```

Ask whether a file is attached

Same behavior, an explicit boundary.
Migrate callers incrementally; extraction is under analysis.

Kaigi on Rails 2026

Image Operations: Wrap the API



Incompatibilities: Investigated

- We have **investigated the incompatibilities** in the CarrierWave upgrade
- The next challenge is **how to roll it out safely**
- Understanding the differences is the starting point

The Same Rollback Strategy Won't Fit

- For file paths, we switched **writes and reads separately**
- A library upgrade changes the underlying implementation
- We cannot reuse the **same flag-based rollback strategy**

Reuse What We Learned

- **AI-assisted QA** may help check compatibility
- **Gradual rollout** may help limit the impact
- Build on our **file-path migration experience**

To be updated.

パス固定で得た品質管理のノウハウを活かして、更新に取り組みたい

Kaigi on Rails 2026

Address Both Upgrade Blockers

1. **Tight coupling → clearer boundaries**
   - Wrap library-specific behavior; migrate callers gradually
2. **Incompatibilities → a safe upgrade plan**
   - Investigation done; adapt our QA and rollout experience

密結合→境界づくり。非互換性については調査はできていて、その上で品質管理しつつ作業

Kaigi on Rails 2026

6. Upgrade or Replace?

What Could Replace CarrierWave?

If we replace CarrierWave...

ActiveStorage

Fit with our data model?

8 attachments / model

OR

Shrine

GCS integration?

Community extension

OR

In-house

Build and maintain?

Tailored to our needs

Of course, we could keep upgrading CarrierWave.

Kaigi on Rails 2026

No Decision Yet

- Upgrading CarrierWave and replacing it are both options
- Catching up to a newer version may be the practical next step
- First, **simplify the behavior we must preserve**
- Keep application needs separate from library internals



Whatever the Choice, Make Responsibilities Clear.

結論はまだ出ていないが、更新にせよ、移行にせよ、
ファイルアップロードの責務と影響範囲を明確にすることが重要。

7. Takeaways



First, split the problem by quality attribute.

まず、課題を性質で分割する。



Make important changes reversible.

重要な変更は、必ず戻せるようにする。



Watch for hidden bugs where behaviors interact.

仕様の組み合わせによる、暗黙のバグに注意する。

Extract image complexity step by step.

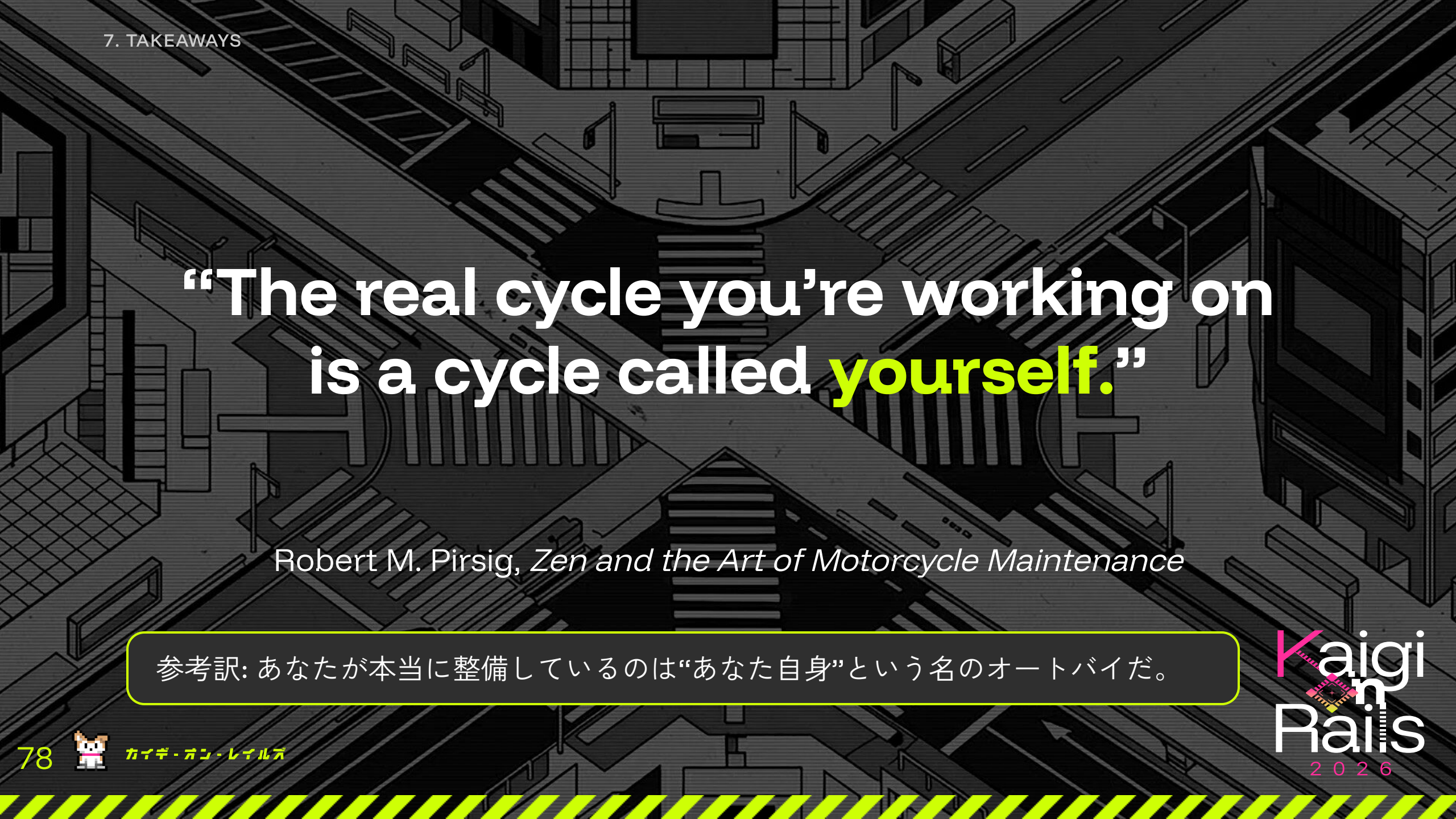
For upgrades or replacement.

画像の複雑な仕様を、少しずつ切り出す。
最終的な更新でも移行でも、その整理が必要になる。



Keep building our quality assurance know-how.

品質管理のノウハウを積み重ねていこう。



“The real cycle you’re working on is a cycle called yourself.”

Robert M. Pirsig, *Zen and the Art of Motorcycle Maintenance*

参考訳: あなたが本当に整備しているのは“あなた自身”という名のオートバイだ。



Face your problems.
Make you and your users happy.

Kaigi on Rails 2026

Thank you!

udzura.jp/slides/2026/kaigionrails

Thanks: Team members! @eminem, @a5-stable, @yudai

カイギ・オン・レイルズ

Kaigi on Rails 2026


Welcome to Hakodate Uh I'm traveling here from Fukuoka, about 100,700 kilometers away, almost a
length of Japan. It's my first time in Hokkaido uh I'm thrilled to
self.introduce!
★ Uchio Kondo
★ from Fukuoka.rb
★ Member of SmartHR
★ Product Engineer
★ SmartHR is a Platinum Sponsor of RubyKaigi 2026
RubyKaigi 2026
We are Hiring!
RubyKaigi 2026